



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 – Estructuras de Datos y Algoritmos

2025 - 2

Tarea 3

Fecha de entrega código: 21 de noviembre, 23:59 Chile continental.

Link a generador de repos: [CLICK AQUÍ](#)

Objetivos

- Identificar la estrategia algorítmica más eficiente según el problema.
- Modelar y resolver problemas utilizando grafos y algoritmos de grafos.
- Incorporar estructuras de datos para optimizar el rendimiento de los algoritmos.

Introducción



Gru está por lograr sus objetivos. Tan solo necesita lidiar con los últimos detalles para salirse con la suya. Para asegurar el éxito de estos últimos pasos decide contratarte una vez más, confiando plenamente en tus habilidades como programador. Esta vez, los desafíos que deberás enfrentar no se resolverán con simples trucos o fuerza bruta. Cada problema requerirá que analices la situación con cuidado y elijas la estrategia adecuada. Gru confía en que podrás mantener la cabeza fría bajo presión y encontrar la mejor forma de ejecutar su plan.

Parte 1: ¡Bombardeando a Vector!



Introducción

Gru se ha enterado de que Vector logró descifrar sus mensajes de la tarea anterior, y está furioso... ¡Pero por suerte no se dio cuenta de que es tu culpa!

Con todos los preparativos listos, finalmente Gru da la orden final: activar el arma definitiva para derrotar a Vector. Pero el dispositivo no es una pieza única, sino que está compuesta por cientos de miles de piezas más pequeñas, tantas piezas que cada una está compuesta por piezas aún más pequeñas, al punto en que podrías encontrar partes del tamaño de un átomo. Tu nuevo trabajo es indicarle a Gru cómo ensamblar las armas de tal forma que se forme correctamente el arma definitiva.

Problema

Recibirás una lista de **Dispositivos**, y por cada dispositivo tendrás una lista de **Partes** necesarias para ensamblarlo, que pueden ser otros dispositivos o materiales. Además, recibirás una lista de **Materiales**. Debes imprimir en orden alfabético todos los dispositivos que puedes crear con los materiales disponibles.

Consideraciones Generales

- $N = |\text{dispositivos}|$
- Los strings tendrán un tamaño máximo de 50 caracteres.
- Los strings tendrán sólo caracteres alfabéticos en minúscula (sin incluir la $\tilde{n}$)
- No habrán dispositivos entre los materiales.
- Cada lista de partes no tiene valores duplicados.
- Puedes necesitar un dispositivo para armar otro.
- Siempre podrás armar al menos un dispositivo.

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `assembly` que se ejecuta con el siguiente comando:

```
./assembly input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando `valgrind` deberás utilizar el siguiente comando:

```
valgrind ./assembly input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

Primero recibirás el entero N , que indica la cantidad de dispositivos, seguido de N dispositivos. Luego recibirás N líneas donde cada una tendrá primero un entero P que indica el número de partes a recibir, y luego las P partes. Finalmente, recibirás una línea con el entero M y luego M materiales.

(rellenar con ejemplo concreto)

input									
1	4	Banana-Blaster	Laser-Pants	Snack-Cannon	Zap-Zooka				
2	3	Snack-Cannon	Banana	Neutrino-Core		# partes del dispositivo 1			
3	1	Uranium-Sheet				# partes del dispositivo 2			
4	2	Titanium	Iron-Sheet			# partes del dispositivo 3			
5	4	Electric-Quark	Dark-Matter	Neutrino-Core		# partes del dispositivo 4			
6	5	Neutrino-Core	Banana	Titanium	Iron-Sheet	Aerogel			

Output

Deberás imprimir todos los dispositivos que puedes ensamblar, ordenados alfabéticamente.

output	
1	Banana-Blaster
2	Snack-Cannon

Parte 2: ¡Elige al Dream Team!



Introducción

Ahora que lograste determinar qué dispositivos puedes construir con los materiales disponibles, ha llegado el momento de poner tu plan en marcha. Sin embargo, no todos los minions saben trabajar con estos dispositivos, por lo que te propones una nueva misión: elegir cuidadosamente al mejor equipo de minions ingenieros.

Problema

Debes elegir el mejor equipo de k minions ingenieros. Cada minion tiene cierta velocidad y eficiencia al trabajar, representados cada uno por un entero positivo. El desempeño de un equipo se calcula como la suma de sus velocidades, multiplicado por la menor de sus eficiencias. Tu objetivo es entregar el mejor desempeño posible al elegir k minions.

Importante: Se espera que el problema se resuelva usando estructuras y/o técnicas algorítmicas pertinentes, no a fuerza bruta.

Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **engineers** que se ejecuta con el siguiente comando:

```
./engineers input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados. En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind, deberás utilizar el siguiente comando:

```
valgrind ./engineers input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

Primero recibirás el entero Q que indica la cantidad de consultas que recibirás. Luego, por cada consulta se te entregará lo siguiente:

- Un entero N que indica la cantidad total de minions.
- Una línea con N enteros que corresponden a la velocidad de cada minion.
- Una línea con N enteros que representan la eficiencia de cada minion.
- Un entero K , que corresponde a la cantidad de minions que puedes elegir para tu equipo.

input	
1	1
2	7
3	10 1 5 9 3 5 2
4	2 10 5 3 4 4 8
5	3

En este ejemplo, el minion 1 tiene velocidad 10 y eficiencia 2; el minion 2 tiene velocidad 1 y eficiencia 10, y así sucesivamente.

Output

Debes imprimir el máximo desempeño posible al elegir k minions. El output para el ejemplo anterior sería:

input	
1	57

Esto resulta de elegir a los minions con las siguientes velocidades y eficiencias:

- Minion 3 Velocidad: 5 Eficiencia: 5
- Minion 4 Velocidad: 9 Eficiencia: 3
- Minion 6 Velocidad: 5 Eficiencia: 4

Entonces el desempeño del equipo es corresponde a: $(5 + 9 + 5) \times \min(5, 3, 4) = 19 \times 3 = 57$

Parte 3: Baby Distance



Introducción

Históricamente, ha sido un problema para Gru encargarse de las nuevas generaciones de Minions, especialmente cuando aún son bebés que están aprendiendo a leer y escribir. Por curiosidad, revisas qué clase de errores cometen los minions bebés y quedas totalmente atónito. . . ¡¡¡No saben nada!!!

Por compasión, decides crear una herramienta que les permita a los cuidadores de los minions bebés recibir feedback inmediato sobre su aprendizaje, indicando cuánto es el factor de error en las palabras que escriben, o visto de otra forma, qué tan lejos estuvieron de haber escrito la palabra correcta.

Problema

La *distancia de edición* (también conocida como Distancia de Levenshtein) entre dos palabras u y v , denotado como $ed(u, v)$, es la mínima cantidad de operaciones necesarias para transformar la palabra u en la palabra v . Dado $u = a_1a_2 \cdots a_n$ las operaciones básicas para calcular la distancia de edición son las siguientes:

$$\text{Sub}(u, i, b) = a_1a_2 \cdots a_{i-1}ba_{i+1} \cdots a_n \quad (\text{sustituye la letra en la posición } i \text{ por } b)$$

$$\text{Del}(u, i) = a_1a_2 \cdots a_{i-1}a_{i+1} \cdots a_n \quad (\text{elimina la letra en la posición } i)$$

$$\text{Ins}(u, i, b) = a_1a_2 \cdots a_ib a_{i+1} \cdots a_n \quad (\text{inserta una letra } b \text{ en la posición } i + 1)$$

Ejemplo

Tenemos que $ed(\text{casa}, \text{asado}) = 3$ puesto que:

$$ins(\text{casa}, 4, d) = \text{casad}$$

$$del(\text{casad}, 1) = \text{asad}$$

$$ins(\text{asad}, 4, o) = \text{asado}$$

Bloques

En la definición anterior, asumimos que cada operación tiene coste 1 y además trabaja sobre un solo carácter. Pero esto no es tan conveniente en la práctica, por lo que podemos extender el problema de *distancia de edición* incorporando el concepto de *bloques*. Un bloque nos permite hacer una secuencia de **inserciones** o **eliminaciones** en varias posiciones continuas, con un costo asociado. La fórmula para calcular el costo total de realizar una misma operación sobre un bloque de tamaño k es la siguiente:

$$\text{cost}(k) = g_o + (k-1)g_e$$

Donde g_o representa el costo de la primera operación realizada, mientras que g_e es el costo asociado a cada repetición de la primera operación sobre posiciones que formen un bloque contiguo. Notemos que si en algún momento hacemos una operación distinta, esa pasaría a ser la nueva *primera operación*.

Por ejemplo, sea $g_o = 5$ y sea el costo por extensión $g_e = 1$. Si queremos eliminar en una palabra 3 letras seguidas, lo óptimo sería usar un bloque de largo 3, el cual tiene un costo de $cost(3) = 5 + (3 - 1) \times 1 = 7$, mientras que eliminar cada letra individualmente nos implicaría una operación de costo 15.

Objetivo

Siendo g_o el costo de realizar una inserción o una eliminación (abrir un bloque), y g_e el costo de extender un bloque y palabras u y v , retornar la distancia de edición $ed(u, v)$

Consideraciones generales

- Las operaciones por bloque no se aplican para las substituciones, estas se hacen de manera individual como en la distancia de edición clásica y tienen un costo de g_o
- Las operaciones por bloque tienen que aplicarse sobre caracteres **contiguos**.
- $ed(u, v) = 0$ si es que $u = v$
- Las palabras tendrán un tamaño máximo de 256 caracteres. Por lo que pueden ser almacenadas en un buffer de ese largo. Y siempre tendrán al menos un carácter.

Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **distance** que se ejecuta con el siguiente comando:

```
./distance input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind, deberás utilizar el siguiente comando:

```
valgrind ./distance input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

Recibirás una primera línea con 3 enteros g_0 , g_e y Q , que representa la cantidad de consultas a recibir.

Luego, recibirás Q líneas que contienen dos palabras u y v para las cuales debes calcular su distancia $ed(u, v)$

input	
1	g_o g_e Q
2	u_1 v_1
3	u_2 v_2
4	.
5	.
6	.
7	u_q v_q

Ejemplo

input	
1	5 1 3
2	casa asado
3	banana ban
4	minion minino

input	
1	11
2	7
3	10

1. En el primero caso se puede eliminar la primera c (bloque de eliminación de largo 1, coste 5) y luego insertar do (bloque de inserción de largo 2, coste 5+1). Coste total 11.
2. En el segundo caso, sólo eliminamos ana por lo que abrimos un bloque de eliminación de largo 3, coste $5 + 2 = 7$
3. En el tercer caso, sustituir o por n (quedando la palabra $mininn$) y luego sustituir la última n por una o . Hacemos dos sustituciones, por lo que el costo es $5 + 5 = 10$.

Informe

Además del código de la tarea, se debe redactar un **breve** informe que explique cómo se pueden implementar algunas estructuras y algoritmos del enunciado. Les recomendamos **comenzar la tarea escribiendo este informe**, ya que pensar y armar un esquema antes de pasar al código ayuda a estructurarse al momento de programar y adelantar posibles errores que su solución pueda tener.

Este informe se debe entregar en un archivo PDF escrito usando LaTeX junto a la tarea (por lo cual tienen la misma fecha de entrega), con nombre `informe.pdf` en la raíz del repositorio. En este se debe explicar al menos los siguientes puntos:

- Parte 1
 1. Explica y justifica cómo modelaste y solucionaste el problema.
- Parte 2
 1. Explica en detalle la estrategia utilizada. Justifica por qué esta estrategia es útil y correcta.
- Parte 3
 1. Explica que condiciones cumple el problema de distancia de edición para que pueda ser resuelto con la técnica de programación que elegiste.
 2. Argumenta por qué extender la distancia de edición con bloques mantiene las condiciones que permiten resolver el problema.
- Bibliografía: Citar código de fuentes externas.

IMPORTANTE: Para asegurar una mejor comprensión y evaluación del informe, es **esencial** que cites el código al que haces referencia. Cada vez que menciones un fragmento de código, incluye el nombre del archivo y el número de línea correspondiente y su *hipervínculo* correspondiente. Puedes aprender cómo crear un *permalink* a tu código consultando la [documentación de GitHub](#). Esto facilitará la revisión y garantizará que el informe sea coherente con la implementación.

Ejemplo:

Si en la Parte 1 del informe estás explicando una función que implementa una determinada operación, la referencia al código podría ser así:

En nuestra implementación, utilizamos una lista enlazada para manejar la estructura de datos, como se muestra en la función `insertar_elemento` ([src/estructuras.c](#), línea 45). Esta elección permite...

De esta manera, aseguras que los revisores puedan localizar rápidamente el fragmento de código relevante y verificar que la explicación sea consistente con la implementación.

Nota: Si por alguna razón no lograste completar alguna parte del código mencionado en el informe, **aún puedes obtener puntaje** en la sección correspondiente. Para ello, debes describir claramente cuál era tu idea de implementación, los pasos que planeabas seguir, y explicar por qué no pudiste completarlo. Esto demuestra tu comprensión del problema y del proceso, lo cual también es valioso para la evaluación.

Finalmente, **puedes** referenciar código visto en clases sin necesidad de incluir el código o pseudocódigo en el informe.

Puedes encontrar un [template en Overleaf](#) disponible en este enlace para su uso en la tarea.

No se aceptarán informes de más de tres páginas (sin incluir bibliografía), informes ilegibles, o generados con inteligencia artificial.

Evaluación

La nota de tu tarea es calculada a partir de testcases de input/output, así como un informe escrito en LaTeX que explique un modelamiento sobre cómo abarcar el problema, las estructuras de datos a utilizar, etc.

La ponderación se descompone de la siguiente forma:

Informe (20 %)	Nota entre partes (70 %)	Manejo de memoria (10 %)
6 % Parte 1	15 % Tests Parte 1	5 % Sin leaks de memoria
7 % Parte 2	25 % Tests Parte 2	5 % Sin errores de memoria
7 % Parte 3	30 % Tests Parte 3	

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 5 segundos** y utilizar menos de 1 GB de RAM¹. De lo contrario, recibirás 0 puntos en ese test.

Recomendación de tus ayudantes

Estas tareas generalmente requieren de mucha dedicación, por lo que desde ya te recomendamos distribuir tu tiempo considerando los plazos definidos. Así mismo, te recomendamos fuertemente que antes de empezar a programar tu tarea, **leas el enunciado detenidamente y con anticipación para que te dediques a entender de manera profunda lo que te pedimos**. Una vez hayas comprendido el enunciado, dedica el tiempo que sea necesario para la planificación, modelación y elaboración de tu informe, para posteriormente poder programar de manera más eficiente. No olvides compilar el código como se indica en la tarea y de preguntar cualquier duda o por problemas que te surjan en [Discussions](#). Estos son consejos de tus ayudantes que te pueden ayudar a pasar el ramo.

Entrega

Código: GIT - Rama principal del repositorio asignado, que debes generar con el link dado al principio de este enunciado. Se entrega a más tardar el día de entrega a las 23:59 hora de Chile continental.

Atraso: Esta tarea considera la política de atraso y cupones especificada en el programa del curso.

Integridad académica

Este curso se adscribe al Código de Honor establecido por la Escuela de Ingeniería. Todo trabajo evaluado en este curso debe ser hecho **individualmente** por el alumno y **sin apoyo de terceros**. Se espera que los alumnos mantengan altos estándares de honestidad académica, acorde al Código de Honor de la Universidad. Cualquier acto deshonesto o fraude académico está prohibido; los alumnos que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada alumno conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Pregrado de la Escuela de Ingeniería.

Uso de IA

Se prohíbe el uso de herramientas de inteligencia artificial para la realización de esta tarea. Esto incluye, pero no se limita a, el uso de modelos de lenguaje como ChatGPT, Copilot, u otras tecnologías similares para la generación automática de código, soluciones, o cualquier parte del trabajo requerido. Nos reservamos el derecho de revisar todas las tareas entregadas con el fin de detectar el uso de inteligencia artificial en la creación de las soluciones. Las tareas en las que se determine que se ha utilizado IA serán consideradas como una infracción a la política de honestidad académica y serán tratadas como casos de copia.

¹Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`